



Capítulo 13 - Sonorização da ferrovia

Associar sons aos acontecimentos da maquete, sem repetir comandos desnecessários nem interromper a automação.

Introdução	A sonorização acrescenta ambiente e movimento à ferrovia. Neste capítulo, vamos organizar arquivos de áudio, escolher quando reproduzi-los e controlar repetições.
Objetivo	Relacionar eventos a sons, distinguir reproduzidor, amplificador e alto-falante e criar uma lógica simples de reprodução.
Meta de aprendizagem	Ao final, você deverá conseguir selecionar um som por região, impedir sobreposição no mesmo reproduzidor e bloquear repetições muito próximas.

1. Que sons podem fazer parte da ferrovia?

Podemos utilizar sons de ambiente, como movimentação na estação, animais no curral ou ruído de circulação próximo ao túnel. Também podemos associar um som curto a um acontecimento, como a chegada de uma composição.

O objetivo é criar uma impressão coerente com a cena. Sons muito altos ou acionados repetidamente podem competir entre si e prejudicar a operação.

Um som de chegada pode começar quando o programa reconhece um evento de passagem. Ele não precisa reiniciar a cada leitura enquanto o sensor permanece acionado.

A sonorização cenográfica não deve ser a única forma de informar uma emergência. Os avisos operacionais precisam continuar perceptíveis mesmo com o áudio de ambiente desativado.

2. As três partes do sistema de áudio

Elemento	Responsabilidade
Reproduzidor ou fonte de áudio	Ler o arquivo e produzir o sinal de áudio.
Amplificador	Fornecer potência para o alto-falante a partir do sinal de áudio.
Alto-falante	Transformar o sinal elétrico em som.

O PAM8403 é um amplificador: ele não guarda nem lê arquivos MP3. Para reproduzir gravações, precisa receber o áudio de uma fonte compatível.



A ESP32 pode enviar comandos a um módulo reproduzidor ou utilizar uma solução própria de áudio com os componentes adequados. O exercício deste capítulo simula a escolha e o controle; não configura uma saída de áudio da ESP32.

3. Regiões sonoras da EFMN 74

As identificações dos alto-falantes ajudam a organizar os sons por cena:

Identificação	Região	Exemplo de som
AF1	Estação	Ambiente da estação ou anúncio.
AF2	Túnel	Passagem de trem.
AF3	Curral	Animais e ambiente rural.
AF4	Pontilhões	Passagem sobre estrutura.
AF5	Chiqueirinho	Ambiente da cena.
AF6	Paredes	Ambiente do trecho.
AF7	Rotunda	Atividade e movimentação local.

AF1 e AF7 estão relacionados à gaveta GA1; AF2 e AF6, à gaveta GA2; AF3, AF4 e AF5, à gaveta GA3. A escolha do reproduzidor e da distribuição do áudio será uma etapa própria de montagem.

A identificação de uma região no programa não direciona fisicamente o som para seu alto-falante. Para isso, é necessário definir a ligação dos canais ou o sistema de seleção de saídas.

4. Uma opção de reproduzidor de arquivos

O DFPlayer Mini é um exemplo de módulo que lê arquivos de um cartão microSD e recebe comandos por comunicação serial. Ele é apresentado como uma opção didática, sem alterar os componentes já escolhidos para a ferrovia.

Para utilizar o comando `playMp3Folder()` da biblioteca `DFRobotDFPlayerMini`, organize os arquivos em uma pasta `mp3` na raiz do cartão, com nomes de quatro algarismos:

Arquivo	Som de exemplo
mp3/0001.mp3	Estação.
mp3/0002.mp3	Túnel.
mp3/0003.mp3	Curral.



Esse padrão é específico da opção apresentada. Outros reprodutores podem exigir outro formato de cartão, nomes ou comandos.

Comece com arquivos curtos e volume baixo. Confira cada gravação antes de associá-la a um evento da automação.

5. Reprodutor e PAM8403

No DFPlayer Mini, as saídas DAC_L e DAC_R fornecem áudio para uma entrada de amplificador compatível. As saídas SPK1 e SPK2 são destinadas à ligação direta de um alto-falante conforme a documentação do módulo.

Para alimentar a entrada de um PAM8403, utilize uma saída de áudio apropriada, não as saídas já amplificadas SPK1/SPK2. Confira os bornes, os níveis e o controle de volume da placa utilizada.

Os alto-falantes de $8\ \Omega$ / 3 W previstos no projeto possuem uma especificação de potência admissível. Isso não significa que um PAM8403 alimentado em 5 V fornecerá 3 W a uma carga de $8\ \Omega$.

As saídas de alto-falante do PAM8403 são em ponte. Ligue cada alto-falante somente entre os dois terminais do seu canal. Não una os terminais de saída ao GND e não interligue saídas de canais ou amplificadores diferentes.

A montagem física exige conferir alimentação, polaridade, referência do sinal de entrada e distribuição dos canais. Nenhum alto-falante deve ser ligado diretamente a um GPIO da ESP32.

6. Regras de reprodução

No exercício, haverá um único reprodutor simulado e três sons. Aplicaremos quatro regras:

- Reproduzir somente quando um evento for recebido.
- Recusar outra solicitação enquanto o reprodutor estiver ocupado.
- Impedir a repetição do mesmo som antes de oito segundos desde seu início.
- Permitir interromper ou desativar o áudio de ambiente sem alterar a operação ferroviária.

Pedidos recusados não serão guardados em uma fila. Ao terminar um som, o programa não inicia automaticamente aquele que foi recusado.

Em uma arquitetura com vários reprodutores independentes, a ocupação e o controle de repetição precisam ser mantidos para cada um.

7. Simular a duração de um som

O programa considerará cada som ocupado por cinco segundos. Esse tempo é apenas uma simulação, controlada por `millis()`.

Na integração com um reprodutor real, utilize a indicação de ocupado ou o evento de fim de reprodução previsto pelo módulo. A duração real do arquivo não deve ser presumida a partir dos cinco segundos do exemplo.



Quando o som é interrompido, o controle de repetição continua valendo. Isso evita que uma sucessão de eventos reinicie o mesmo arquivo imediatamente.

8. Programa completo

Crie `intermediate_13_sound_control` e utilize somente a ESP32 de bancada conectada por USB. Não são necessários reproduzidor, amplificador, cartão ou biblioteca adicional para a simulação.

Comandos: 1 para estação, 2 para túnel, 3 para curral, S para interromper, M para desativar o áudio e A para ativá-lo novamente. Copie todos os blocos na ordem para o mesmo programa.

```
const int soundCount = 3;
const int trackNumbers[soundCount] = {1, 2, 3};
const char* regionNames[soundCount] = {"Station", "Tunnel", "Corral"};
const unsigned long simulatedDuration = 5000;
const unsigned long repeatInterval = 8000;
bool soundEnabled = true;
bool playerBusy = false;
bool hasPlayed[soundCount] = {false, false, false};
unsigned long lastStarted[soundCount] = {0, 0, 0};
unsigned long playbackStarted = 0;
```

```
void stopPlayback() {
    if (playerBusy) Serial.println("Stop playback requested");
    playerBusy = false;
}
```

```
void requestSound(int index) {
    if (index < 0 || index >= soundCount) return;
    unsigned long now = millis();
    if (!soundEnabled) {
        Serial.println("Sound disabled - request rejected");
        return;
    }
    if (playerBusy) {
        Serial.println("Player busy - request rejected");
        return;
    }
    if (hasPlayed[index] &&
        now - lastStarted[index] < repeatInterval) {
        Serial.println("Repeat interval active - request rejected");
        return;
    }
    hasPlayed[index] = true;
    lastStarted[index] = now;
    playbackStarted = now;
    playerBusy = true;
    Serial.print("Play track: ");
    Serial.print(trackNumbers[index]);
    Serial.print(" - Region: ");
    Serial.println(regionNames[index]);
}
```



```
void updatePlayback() {
    if (playerBusy &&
        millis() - playbackStarted >= simulatedDuration) {
        playerBusy = false;
        Serial.println("Simulated playback completed");
    }
}

void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("EFMN 74 - Sound control simulation");
    Serial.println("Station: 1 | Tunnel: 2 | Corral: 3");
    Serial.println("Stop: S | Disable: M | Enable: A");
}

void loop() {
    updatePlayback();
    if (Serial.available() > 0) {
        char command = Serial.read();
        if (command >= '1' && command <= '3') {
            requestSound(command - '1');
        } else if (command == 'S') {
            stopPlayback();
        } else if (command == 'M') {
            stopPlayback();
            soundEnabled = false;
            Serial.println("Ambient sound disabled");
        } else if (command == 'A') {
            soundEnabled = true;
            Serial.println("Ambient sound enabled - waiting for event");
        } else if (command != '\n' && command != '\r') {
            Serial.println("Invalid command");
        }
    }
}
```

9. Entendendo o controle

trackNumbers relaciona cada região a um arquivo. playerBusy representa a ocupação do único reproduzido simulado.

hasPlayed diferencia um som que nunca foi reproduzido de outro que está no intervalo de repetição. Assim, o primeiro evento não precisa aguardar oito segundos após a inicialização.

lastStarted guarda o início de cada som separadamente. A comparação de tempo utiliza unsigned long e subtração, como nos capítulos anteriores.

updatePlayback() acompanha a duração sem bloquear o programa. Enquanto o som está ocupado, os comandos S e M continuam sendo reconhecidos.

Ativar o áudio com A não reinicia um arquivo interrompido nem reproduz pedidos recusados. É necessário receber um novo evento.

Exercício prático

Meta: conferir seleção, ocupação, intervalo de repetição e controle de áudio.



1. Selecione a placa e a porta, compile e envie o programa.
2. Abra o Monitor Serial em 115200 baud e pressione EN ou RESET.
3. Envie 1 e confira a mensagem de reprodução da estação.
4. Antes de cinco segundos, envie 2. Confira a recusa por reprodutor ocupado.
5. Aguarde a mensagem de fim da reprodução simulada.
6. Envie 1 antes de completar oito segundos desde o primeiro início. Confira a recusa por intervalo de repetição.
7. Depois dos oito segundos, envie 1 novamente. A reprodução deverá ser aceita.
8. Envie S durante a reprodução. Confira a interrupção.
9. Envie 3 e depois M. Confira a interrupção e a desativação do áudio.
10. Envie 2 com o áudio desativado. O pedido deverá ser recusado.
11. Envie A. Nenhum som deverá começar automaticamente.
12. Envie um novo comando permitido e confira a reprodução.

Exemplos de mensagens:

```
Play track: 1 - Region: Station  
Player busy - request rejected  
Simulated playback completed  
Repeat interval active - request rejected
```

Esse teste não produz som: ele valida a sequência lógica antes da escolha e montagem do circuito de áudio.

10. Ligação com os eventos da ferrovia

Um evento de chegada reconhecido pela lógica da operação pode chamar `requestSound(0)`. Um evento de passagem no túnel pode chamar `requestSound(1)`.

A chamada deve ocorrer no reconhecimento do evento, e não continuamente enquanto um Hall permanecer acionado. A lógica ferroviária continua responsável por interpretar o evento e verificar sua coerência.

Falhar ao reproduzir um som ambiental não deve liberar uma rota, alterar um desvio ou remover um bloqueio de tração. O áudio é uma tarefa auxiliar.

Se um aviso operacional precisar interromper o ambiente, defina uma prioridade explícita e mantenha um canal de indicação adequado. A desativação da sonorização cenográfica não deve desativar o alarme da emergência.

11. Adaptação para um reprodutor real

No DFPlayer Mini, após configurar a comunicação, iniciar a biblioteca e conferir o cartão, os comandos da biblioteca podem reproduzir ou interromper arquivos:



```
// The player object must already be initialized.  
player.volume(10);  
player.playMp3Folder(1);  
  
// When a stop request is received:  
player.stop();
```

Esse trecho ilustra a adaptação e não é um programa completo. O volume 10 é um ajuste inicial baixo na escala de 0 a 30 do módulo; o volume adequado depende do arquivo, do amplificador e do alto-falante.

A integração deverá tratar falha de inicialização, arquivo ausente, indicação de ocupado e fim da reprodução. O estado `playerBusy` deve refletir o reprodutor real, em vez do tempo simulado.

Confira os níveis da comunicação serial antes de conectá-los à ESP32. A alimentação de 5 V de um módulo não significa que suas conexões de sinal possam aplicar 5 V a um GPIO.

Comece com uma região e um arquivo. Depois, amplie a distribuição sonora entre as gavetas GA1, GA2 e GA3, registrando fonte de áudio, canal, alto-falante, volume e evento de acionamento.

Resumo do capítulo

- O reprodutor lê arquivos; o amplificador fornece potência; o alto-falante produz som.
- O PAM8403 precisa de uma fonte de áudio.
- Uma identificação de região não seleciona fisicamente uma saída.
- O acionamento deve ocorrer por evento, sem reiniciar a cada leitura do sensor.
- Ocupação e intervalo de repetição evitam sobreposição e insistência.
- `millis()` permite acompanhar o áudio sem bloquear a automação.
- Ativar novamente o ambiente não inicia sons automaticamente.
- A sonorização ambiental não substitui os avisos de segurança.

Conclusão do Curso Intermediário: os conceitos de funções, módulos, estados, eventos e permissões permitem organizar a integração progressiva da automação da ferrovia. A aplicação definitiva depende da montagem, dos testes e da validação de cada função.

Referências técnicas

DFRobot: DFPlayer Mini e biblioteca DFRobotDFPlayerMini. <https://wiki.dfrobot.com/dfr0299>

Diodes Incorporated: PAM8403, amplificador de áudio estéreo.
<https://www.diodes.com/part/view/PAM8403>